

Tecnologias de Machine Learning para Detecção de Intrusões em Redes de Computadores: Uma Pesquisa Exploratória e Experimental

¹ **Lucas Kerr do Amaral**, ² **Marcela Nalesso**

¹ *Ciência da Computação – FEI* ² *Ciência da Computação – FEI*

¹ *lamaral2002@gmail.com*, ² *marcelanalesso@gmail.com*

Orientador: Prof. Dr. Leonardo Anjoletto Ferreira laferreira@fei.edu.br

Resumo: Diante da insuficiência performática em métodos tradicionais, este artigo apresenta o desenvolvimento parcial de um sistema de detecção de intrusões em redes. As etapas incluem: i) coleta e preparação de dados com bases de ataques; ii) redução de dimensionalidade via *Principal Component Analysis* (PCA) e Matriz de Correlação; iii) análise exploratória para distinguir tráfego normal, ataques e falsos positivos; iv) implementação e avaliação preliminar de modelos de machine learning para classificação de intrusões. Resultados preliminares serão apresentados. Como contribuições futuras, prevê-se aprimoramento dos algoritmos, ajuste de hiperparâmetros à novos modelo, e evolução da sistema para tempo real e em cenários reais.

Palavras-chaves: Sistemas de Detecção de Intrusões, Machine Learning, Segurança da Informação, Análise de Dados, Redução de Dimensionalidade e Cybersegurança.

I. Lista de Abreviaturas e Siglas

DoS *Denial of service* ataque que consiste em negar o serviço para outros acessos.

R2L *Remote to local* escalação de serviço de um acesso remoto para permissão local.

U2R *User to Root* escalação de privilégios de usuário com baixo nível de permissão para usuário com nível administrativo.

IDS *Intrusion Detection System* Sistema de detecção de intrusão.

HIDS *Host based Intrusion detection System* Sistemas *IDS* baseados em host.

NIDS *Network based Intrusion detection System* Sistemas *IDS* baseados em tráfego de rede.

Zero-Day Ataques onde se tem o conhecimento a zero dias (ataques inéditos).

ML *Machine Learning* algoritmos de aprendizado de máquina.

PCA *Principal Component Analysis* análise de componentes principais.

SVM *Support Vector Machine* algoritmo de *ML* baseado em um vetor de suporte para classificação.

IoT *Internet of Things* Conceito de integração de sistemas embarcados.

5G Redes móveis de alta velocidade.

CART *Classification and Regression Tree* Modelo de árvore de decisão para classificação e regressão.

One-Class SVM algoritmo de *SVM* não supervisionado utilizado para detecção de anomalias.

NSL-KDD Base de dados estudada neste artigo.

ADFA *Intrusion Detection Systems Dataset*.

ID3 *Iterative Dichotomizer* Algoritmo utilizado para gerar árvores de decisão.

C4.5 Algoritmo utilizado para gerar árvores de decisão, sucessor do *ID3*.

SOM Mapas auto-organizáveis.

Overfitting Situação onde a árvore fica desbalanceada.

MultiTree Algoritmos baseados em múltiplas árvores.

REPTree Algoritmo de árvore de decisão rápida.

stepwise Método automático de seleção de variáveis preditoras.

Apache Spark Software para processamento distribuído.

AUROC Métrica para avaliar desempenho de modelos de classificação binária.

Probe Ataques de mapeamento de informações sobre o alvo.

StandardScaler Ferramenta de pré-processamento para padronizar atributos.

Decision Tree Algoritmo de árvore de decisão.

Random Forest Algoritmo que cria diversas árvores de decisão e vota entre valores em comum.

KNN *K Nearest-Neighbor* Método de *ML* supervisionado para classificação e regressão.

Logistic Regression Método de *ML* que calcula a regressão logística dos atributos.

LoadModules Carregamento de módulos de linguagens como *perl* para uso indevido.

Perl Linguagem de programação.

nmap Software de análise e mapeamento de rede e host.

II. Introdução

O avanço das redes de computadores e a crescente digitalização de serviços têm ampliado significativamente a exposição de sistemas a ameaças cibernéticas. Esse crescimento é acompanhado por um aumento expressivo no volume de dados trafegados, que passou de aproximadamente 16 GB por usuário ao mês em 2017 para cerca de 50 GB mensais em 2022 [22]. Esse cenário contribui diretamente para a ampliação da superfície de ataque, refletindo no aumento da quantidade e da sofisticação de ameaças, como negações de serviço, varreduras de rede e acessos não autorizados. Paralelamente, os impactos de falhas de segurança tornam-se cada vez mais críticos, sendo que mais de 14 bilhões de registros de dados foram vazados desde 2013, além de prejuízos financeiros significativos, como os 29,8 bilhões de dólares perdidos em golpes telefônicos apenas no ano de 2020 [4]. Diante desse contexto, a área de Segurança da Informação destaca-se como fundamental para garantir a integridade, confidencialidade e disponibilidade das informações.

Os Sistemas de Detecção de Intrusão (IDS) são fundamentais para a segurança de redes, pois monitoram atividades suspeitas e identificam possíveis ataques por meio da análise de tráfego e comportamento. Esses sistemas podem ser implementados em hardware ou software e realizam funções como geração de alertas e resposta a incidentes. O conceito de IDS surgiu na década de 1980, inicialmente baseado na análise de logs e padrões conhecidos, o que limitava sua eficácia frente a ameaças mais complexas. Com o avanço dos ataques, os IDS evoluíram para abordagens mais dinâmicas, incorporando análise comportamental e, posteriormente, técnicas de detecção por anomalias na década de 1990, permitindo a identificação de ataques desconhecidos [23]. Dessa forma, os IDS consolidaram-se como mecanismos essenciais para a proteção de ativos digitais e mitigação de ameaças em redes [1].

Nesse sentido, o uso de técnicas de *Machine Learning* tem se consolidado como uma alternativa promissora [14]. Esses métodos permitem a identificação de padrões complexos nos dados, possibilitando a detecção de comportamentos anômalos sem depender exclusivamente de regras pré-definidas. No entanto, a eficácia desses modelos depende diretamente da qualidade dos dados utilizados, da seleção adequada de atributos e da capacidade de lidar com a alta dimensionalidade das bases. Assim, compreender a influência dessas variáveis torna-se essencial para o desenvolvimento de soluções mais eficientes.

Dessa forma, este trabalho tem como objetivo geral analisar e avaliar técnicas de *Machine Learning* aplicadas à detecção de intrusões, identificando atributos relevantes e comparando o desempenho de algoritmos de classificação. Especificamente, propõe-se:

- Realizar a análise exploratória na base NSL-KDD, para compreender sua estrutura, identificar características gerais dos dados e verificar a presença de inconsistências, valores ausentes ou desbalanceamento entre classes.
- Investigar a distribuição das classes e identificar padrões nos dados, buscando compreender o comportamento do tráfego de rede e possíveis distinções entre atividades normais e diferentes tipos de ataques.
- Treinar modelos de classificação baseados em *Machine Learning*, aplicando diferentes algoritmos com o objetivo de identificar padrões de intrusão e classificar o tráfego de rede.

Os resultados obtidos servirão como base para o desenvolvimento de um protótipo de sistema de detecção de intrusão, contribuindo para o avanço de soluções mais eficazes na área.

III. Fundamentação Teórica

A segurança da informação tornou-se um elemento central na sociedade contemporânea devido à crescente dependência de sistemas digitais. Com a consolidação da chamada “civilização cibernética” [3], iniciada a partir da revolução digital no final do século XX [21], houve um aumento significativo na produção e circulação de dados, ampliando a superfície de ataque para agentes maliciosos. Nesse cenário, a segurança mostra-se indispensável, sobretudo pelo uso intensivo de sistemas digitais em setores críticos como bancos, saúde, governo e educação, nos quais a proteção das informações é fundamental para garantir a continuidade dos serviços, a confiabilidade das operações e a segurança da sociedade como um todo [3].

É desta forma que o foco central da área de pesquisa é proteger dados contra ameaças internas e externas, assegurando a tríade da confidencialidade, integridade e disponibilidade (CIA) de forma eficiente e com o menor custo possível [11]. Nesse contexto, evidencia-se que ataques cibernéticos representam riscos significativos para as organizações, podendo ocasionar perdas financeiras expressivas, danos à reputação institucional e até impactos críticos, inclusive fatais, quando afetam infraestruturas essenciais [3]. Além disso, observa-se que organizações estão constantemente sob ataques direcionados, o que reforça a complexidade e a frequência dessas ameaças no ambiente digital contemporâneo [11]. Dessa forma, torna-se imprescindível a adoção de estratégias robustas de segurança da informação, capazes de mitigar riscos e garantir a proteção contínua dos ativos informacionais diante de um cenário cada vez mais hostil.

As ameaças em redes de computadores apresentam diversidade e complexidade, sendo classificadas em diferentes categorias conforme seu objetivo e forma de atuação. Entre as principais estão os

ataques de negação de serviço (*DoS*), que visam indisponibilizar recursos, ataques de sondagem (*probing*), que coletam informações da rede, além de técnicas de escalonamento de privilégios (*U2R*) e acesso remoto indevido (*R2L*) [3]. Além dessas categorias, destacam-se ameaças modernas como *malware*, *botnets* e ataques de engenharia social, que exploram vulnerabilidades técnicas e humanas [22] [8]. Essa variedade evidencia que os sistemas precisam lidar não apenas com ataques diretos, mas também com estratégias sofisticadas de invasão e evasão. Portanto, a classificação das ameaças é fundamental para orientar mecanismos de detecção e defesa mais eficientes.

Um dos maiores desafios atuais da segurança em redes está relacionado aos ataques de dia zero, que exploram vulnerabilidades ainda desconhecidas pelos sistemas de defesa. Por não possuírem assinaturas previamente registradas, esses ataques são difíceis de identificar por métodos tradicionais, funcionando como ameaças invisíveis até que sejam descobertas [3]. Casos reais, como a Operação Aurora, demonstram o potencial destrutivo dessas ameaças, incluindo roubo de dados sensíveis e comprometimento de infraestruturas críticas [3]. Além disso, o aumento significativo desse tipo de ataque nos últimos anos reforça a necessidade de abordagens mais avançadas de detecção. Dessa forma, a evolução das ameaças exige a adoção de soluções mais inteligentes e adaptativas.

Nesse contexto, os Sistemas de Detecção de Intrusão (*IDS*) desempenham um papel central na proteção de redes, sendo responsáveis por monitorar atividades e identificar comportamentos suspeitos. Esses sistemas permitem a detecção precoce de ataques, possibilitando ações de mitigação antes que danos maiores ocorram. No entanto, ainda apresentam limitações importantes, como altas taxas de falsos positivos e dificuldade na identificação de ataques desconhecidos [1]. Tais limitações evidenciam a necessidade de evolução desses mecanismos, especialmente com o uso de técnicas mais avançadas, como aprendizado de máquina, capazes de analisar padrões complexos e melhorar a eficiência da detecção [23]. Assim, o aprimoramento dos *IDS* é fundamental para enfrentar os desafios contemporâneos da segurança em redes.

Os *IDS* podem ser classificados de acordo com sua forma de implantação e método de detecção, o que influencia diretamente sua eficiência. Quanto à implantação, destacam-se os sistemas baseados em host (*HIDS*), que monitoram atividades locais, e os baseados em rede (*NIDS*), que analisam o tráfego em pontos estratégicos da infraestrutura. Já em relação ao método de detecção, os *IDS* podem operar por meio de assinaturas, identificando padrões conhecidos de ataques, ou por anomalias, detectando desvios em relação ao comportamento considerado normal. Além disso, abordagens híbridas combinam essas técnicas para superar limitações individuais [14] [4] [22] [15].

Essa diversidade de classificações demonstra a necessidade de adaptar os mecanismos de detecção conforme o contexto e os tipos de ameaças enfrentadas. Portanto, a escolha do tipo de *IDS* deve considerar os requisitos específicos do ambiente.

Apesar de sua relevância, os *IDS* ainda apresentam limitações significativas que comprometem sua eficácia. Sistemas baseados em assinatura, por exemplo, são incapazes de identificar ataques desconhecidos, como os de *Zero-Day*, enquanto sistemas baseados em anomalias podem gerar altas taxas de falsos positivos ao interpretar comportamentos legítimos como ameaças [21]. Além disso, muitos *IDS* enfrentam desafios relacionados ao alto custo computacional, datasets mais modernos e à dificuldade de operar em tempo real em ambientes com grande volume de tráfego. Essas limitações evidenciam que, embora essenciais, os *IDS* tradicionais ainda não são totalmente suficientes para lidar com a complexidade das ameaças modernas [1] [23]. Dessa forma, torna-se necessário aprimorar continuamente esses sistemas.

O uso de Machine Learning (*ML*) tem se consolidado como uma abordagem fundamental para aprimorar os Sistemas de Detecção de Intrusão (*IDS*), permitindo a análise automatizada de grandes volumes de dados de rede. Diferentemente dos métodos tradicionais, o *ML* possibilita a identificação de padrões complexos e a detecção de atividades maliciosas sem a necessidade de intervenção humana constante. Isso ocorre porque os modelos são capazes de aprender a partir de dados históricos, tornando os sistemas mais adaptativos e eficientes na identificação de ameaças [4] [22]. Dessa forma, o *ML* amplia significativamente a capacidade dos *IDS* em lidar com ambientes dinâmicos e com grande volume de tráfego. Assim, sua aplicação representa um avanço importante na evolução das soluções de segurança de redes.

As técnicas de Machine Learning aplicadas aos *IDS* podem ser classificadas em diferentes abordagens, sendo as mais comuns o aprendizado supervisionado, não supervisionado e por reforço. No aprendizado supervisionado, os modelos são treinados com dados previamente rotulados, permitindo a classificação do tráfego como normal ou malicioso, enquanto o aprendizado não supervisionado busca identificar padrões ocultos sem a necessidade de rótulos. Além disso, o processo de construção desses modelos envolve a divisão dos dados em conjuntos de treinamento e teste, garantindo a avaliação do desempenho do sistema [4] [1]. Essa diversidade de abordagens permite que os *IDS* sejam adaptados a diferentes cenários e tipos de ataque, aumentando sua flexibilidade e eficiência. Portanto, a escolha da técnica de aprendizado influencia diretamente a eficácia da detecção de intrusões.

Entre os principais benefícios do uso de *ML* em *IDS* está a capacidade de detectar ataques desconhecidos, como os *Zero-Day*, além de lidar com

relações complexas e não lineares presentes nos dados de rede. No entanto, essa abordagem também apresenta desafios relevantes, como o alto custo computacional e o tempo elevado de treinamento. Outro problema recorrente é o desbalanceamento dos dados, que dificulta a detecção de ataques menos frequentes, impactando negativamente o desempenho dos modelos. Essas limitações evidenciam que, embora a *ML* traga avanços significativos, sua aplicação ainda exige cuidados na preparação e tratamento dos dados [4]. Dessa forma, o sucesso do uso de *ML* em *IDS* depende não apenas dos algoritmos, mas também da qualidade dos dados utilizados.

Além disso, etapas como pré-processamento, seleção de atributos e redução de dimensionalidade são essenciais para garantir o desempenho dos modelos de Machine Learning em *IDS*. A limpeza dos dados, normalização e codificação de variáveis contribuem para a construção de modelos mais robustos, enquanto técnicas como seleção de atributos e métodos como *PCA* ajudam a reduzir a complexidade dos dados e melhorar a acurácia. Para avaliar a eficácia dos modelos, são utilizadas métricas como acurácia, precisão, recall e F1-score, que permitem analisar o equilíbrio entre a detecção de ataques e a geração de falsos positivos. Assim, a integração entre boas práticas de tratamento de dados e algoritmos de *ML* é fundamental para o desenvolvimento de *IDS* mais eficientes e confiáveis [4] [22]. Portanto, o uso estruturado dessas técnicas potencializa os resultados obtidos na detecção de intrusões.

A diversidade de algoritmos utilizados em sistemas de detecção de intrusão é destacada no trabalho de [16], que realiza uma revisão abrangente das técnicas de *IDS*, incluindo métodos baseados em Bayes, teoria dos jogos e mineração de dados. O estudo evidencia que algoritmos de mineração, como árvores de decisão, e métodos como *SVM* apresentam altas taxas de acurácia na detecção de ataques. Esses resultados indicam que não existe uma abordagem única dominante, mas sim um conjunto de técnicas complementares que podem ser utilizadas conforme o contexto da aplicação.

A aplicação de algoritmos supervisionados em ambientes modernos é analisada por [19], que investiga o desempenho de diferentes modelos em cenários como *IoT* e *5G*. O estudo compara algoritmos como *CART* e *Random Forest*, destacando que o uso de múltiplas árvores (ensemble) resulta em melhor desempenho. Os resultados mostram que o *Random Forest* atingiu 99,65% de acurácia, superando o *CART* isolado (98%). Esse achado reforça a importância de abordagens híbridas e ensembles para melhorar a robustez dos modelos.

A integração de diferentes algoritmos também é explorada por [15], que propõem um modelo híbrido combinando *C5.0* e *One-ClassSVM*. O estudo utiliza todos os atributos do *NSL-KDD* para treina-

mento e demonstra que a combinação entre detecção de assinatura e anomalia melhora a capacidade do sistema. Os resultados mostram acurácia de 83,24% no *NSL-KDD* e 97,04% no dataset *ADFA*. Esses resultados evidenciam que a combinação de algoritmos pode superar limitações individuais de cada abordagem.

A eficiência e interpretabilidade das árvores de decisão são destacadas no estudo de [4], que propõe esse algoritmo como modelo central para detecção de intrusão. O trabalho explora variações como *ID3*, *C4.5* e *CART*, enfatizando sua estrutura baseada em nós de decisão e ramos. Os resultados mostram que o modelo *CART* alcançou 97,36% de sucesso em aplicações preditivas. Esse desempenho, aliado à facilidade de interpretação, torna as árvores de decisão altamente relevantes para sistemas que exigem transparência nas decisões.

A robustez das árvores de decisão em arquiteturas híbridas é demonstrada por [7], que combinam o algoritmo *C4.5* com mapas auto-organizáveis (*SOM*). O uso de técnicas de poda (pruning) é destacado como estratégia para evitar o *overfitting*. Os resultados mostram que o sistema alcançou 99,84% de precisão na detecção de ataques de uso indevido. Esse resultado evidencia que árvores de decisão, quando bem ajustadas e combinadas com outras técnicas, podem atingir níveis extremamente elevados de desempenho.

A utilização de árvores em modelos ensemble é aprofundada por [10], que propõem o algoritmo *MultiTree* baseado em múltiplas árvores *CART* e votação majoritária. O estudo também aborda o problema de desbalanceamento de dados, comum em *IDS*. Os resultados mostram que o modelo atingiu 85,2% de acurácia em cenários desafiadores, superando abordagens isoladas. Esse achado reforça que múltiplas árvores aumentam a capacidade de generalização dos modelos.

A aplicabilidade das árvores de decisão em cenários de alta performance é evidenciada por [18], que utilizam algoritmos como *C4.5* e *REPTree* em ambientes de Big Data. Os resultados mostram taxas de verdadeiros positivos superiores a 99,9%, indicando alta eficiência na detecção de intrusões em tempo real. Esse resultado demonstra que árvores de decisão são adequadas para sistemas que exigem rapidez e precisão simultaneamente.

Por fim, [2] consolidam, em sua revisão sistemática, que algoritmos baseados em árvores apresentam acurácia variando entre 85,2% e 99,9%, dependendo do contexto e da preparação dos dados. Esses resultados reforçam que as árvores de decisão são uma das abordagens mais consistentes e versáteis dentro da área de detecção de intrusão, especialmente quando combinadas com técnicas de pré-processamento e seleção de atributos.

A relevância do pré-processamento e da redução de dimensionalidade em sistemas de detecção de in-

trusão é evidenciada no estudo de [23], que revisa o estado da arte de *IDS* baseados em inteligência artificial e aprendizado de máquina. O trabalho analisa conjuntos de dados amplamente utilizados, como KDDCup99 e CICIDS2017, destacando o elevado número de atributos (41 e 80, respectivamente) e a necessidade de técnicas como PCA para reduzir essa dimensionalidade. Os resultados mostram que modelos que utilizam essas estratégias, especialmente combinados com deep learning e métodos de ensemble, atingem acurácias entre 94,4% e 99,24%. Esses achados indicam que o pré-processamento não é apenas uma etapa auxiliar, mas um fator determinante para viabilizar o uso de modelos mais complexos em ambientes de *IDS*.

A importância da engenharia de atributos é aprofundada no estudo de [22], que investiga técnicas de seleção de atributos e seu impacto no desempenho dos modelos. O trabalho analisa diferentes abordagens, como filtros, wrappers e métodos embutidos, demonstrando que a redução de conjuntos de dados para 5 a 20 atributos relevantes pode melhorar significativamente a eficiência dos algoritmos. Os resultados indicam que, utilizando técnicas como ganho de informação e qui-quadrado, modelos baseados em árvores de decisão podem alcançar até 95,5% de precisão. Esse resultado reforça que a escolha adequada de atributos reduz a complexidade do problema e aumenta a capacidade discriminativa dos modelos.

A eficiência da redução de atributos em cenários de alta demanda computacional é destacada por [12], que propõem uma abordagem multiestágio para seleção de características em detecção de anomalias. O estudo reduz progressivamente o conjunto de atributos de 41 para 30 e posteriormente para 16, utilizando técnicas de filtro e regressão *stepwise*. Os resultados mostram que a acurácia do modelo aumentou de 75,53% para 78,68% após a primeira redução, mantendo-se praticamente estável mesmo com menos atributos. Esses achados evidenciam que é possível reduzir significativamente a dimensionalidade sem perda de desempenho, o que é especialmente relevante para sistemas em tempo real.

A relação entre seleção de atributos e desempenho em ambientes de Big Data é abordada por [17], que investigam modelos de detecção de intrusão utilizando *ApacheSpark*. O estudo aplica o teste qui-quadrado para selecionar 17 atributos mais relevantes a partir das 41 características do KDD99. Os resultados mostram um *AUROC* de 99,55%, evidenciando alto poder discriminativo mesmo com um conjunto reduzido de atributos. Esse resultado demonstra que a redução de dimensionalidade é essencial não apenas para eficiência computacional, mas também para manter ou até melhorar a qualidade das previsões em ambientes distribuídos.

A importância da seleção de atributos também é reforçada por [18], que investigam sistemas de de-

tecção de intrusão em ambientes de Big Data de alta velocidade. O estudo identifica que apenas 9 atributos são suficientes para alcançar desempenho superior em classificação, utilizando algoritmos como *C4.5* e *REPTree*. Os resultados mostram taxas de verdadeiros positivos superiores a 99,9%, indicando que a redução agressiva de atributos pode ser altamente eficaz quando bem direcionada. Esse achado é particularmente relevante para aplicações em tempo real, onde a latência e o custo computacional são fatores críticos.

Além disso, [1] reforçam, em sua revisão sobre técnicas supervisionadas para *IDS*, que a seleção de atributos é um dos fatores mais críticos para o desempenho dos modelos. O estudo analisa múltiplos datasets e demonstra que a redução para conjuntos de 10 a 19 atributos é comum em abordagens eficientes. Os resultados indicam que modelos como Random Forest podem atingir até 99,9% de acurácia quando combinados com estratégias adequadas de pré-processamento. Esses achados consolidam a ideia de que a qualidade dos dados é tão importante quanto a escolha do algoritmo.

A fim de organizar e comparar os principais trabalhos¹ relacionados à detecção de intrusões e machine learning, foi elaborada a Tabela I encontrada na seção de Anexos contendo os estudos mais relevantes identificados na literatura.

IV. Metodologia

Este trabalho propõe uma abordagem voltada na avaliação do desempenho de algoritmos de aprendizado de máquina supervisionado aplicados à detecção de intrusões em redes de computadores. A metodologia adotada foi estruturada em etapas sequenciais, contemplando desde a seleção das bases de dados até a avaliação dos algoritmos utilizados, conforme ilustrado na Figura 1.

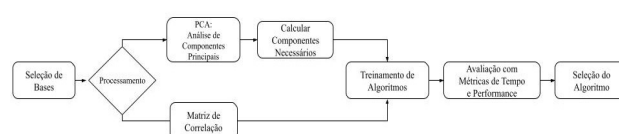


Figura 1. Metodologia do projeto

A pesquisa possui caráter aplicado, com abordagem quantitativa e objetivo experimental, buscando comparar o desempenho de diferentes algoritmos de classificação a partir da utilização de técnicas de redução e seleção de atributos. O fluxo metodológico foi dividido em cinco etapas principais: seleção das bases de dados, processamento dos dados, aplicação de técnicas de redução de dimensionalidade, treinamento dos algoritmos e avaliação de desempenho.

¹Arquivo disponível em: https://1drv.ms/x/c/46f377637be54264/IQB7NxizkmLAS7_X22no62qJAZ115FV9_5Dt7G3kALLXvTY?e=ypAxe1

A. Seleção das Bases de Dados

A base de dados utilizada neste trabalho foi a *NSL – KDD*, uma versão aprimorada da base KDD Cup 1999, desenvolvida com o objetivo de reduzir redundâncias e inconsistências presentes no conjunto original. A escolha dessa base ocorreu devido à sua ampla utilização na literatura científica voltada à detecção de intrusões e aprendizado de máquina, sendo considerada uma das principais referências para avaliação de sistemas *IDS*. A base contém registros de conexões de rede classificados entre tráfego normal (53,46%) e diferentes categorias de ataques (46,54%), possibilitando a aplicação de técnicas de classificação supervisionada.

Os dados foram obtidos por meio da plataforma *Kaggle*², utilizando a biblioteca *KaggleHub*. O desenvolvimento do projeto foi realizado em linguagem *Python*, com execução no ambiente *Jupyter Notebook*³, permitindo a organização e execução sequencial das etapas do experimento. Além disso, foram utilizadas bibliotecas específicas para manipulação, processamento, visualização e análise dos dados, contribuindo para o desenvolvimento completo da metodologia proposta.

Entre as principais bibliotecas utilizadas estão *Pandas*, responsável pela leitura e organização dos dados em estruturas tabulares; *NumPy*, utilizado em operações matemáticas e manipulação numérica; *Matplotlib* e *Seaborn*, empregados na geração de gráficos e visualizações estatísticas; e *Scikit-learn*, utilizada na implementação das técnicas de aprendizado de máquina, incluindo codificação de atributos categóricos, padronização dos dados, aplicação do *PCA*, divisão dos conjuntos de treinamento e teste, treinamento dos algoritmos e validação dos modelos. Essas ferramentas foram fundamentais para o processamento dos dados e análise dos resultados obtidos ao longo da pesquisa.

B. Processamento dos Dados

Após o carregamento da base de dados *NSL – KDD*, foi realizada uma etapa de pré-processamento com o objetivo de preparar os dados para aplicação dos algoritmos de aprendizado de máquina. Inicialmente, foram definidos manualmente os nomes dos 41 atributos da base, permitindo melhor interpretação das variáveis e facilitando a manipulação das informações durante o desenvolvimento do projeto. Em seguida, os dados foram carregados em estruturas do tipo *DataFrame* utilizando a biblioteca *Pandas*, possibilitando operações de análise, filtragem e transformação dos registros. Durante essa etapa, também

foi realizada a verificação da estrutura da base, incluindo quantidade de linhas, colunas, tipos de dados e identificação das classes presentes no conjunto de dados. A coluna *label* foi convertida em binário, classificando como 0 para tráfego normal e 1 para qualquer tipo de ataque (*DoS*, *Probe*, *R2L*, *U2R*). Posteriormente, a coluna denominada *difficulty* foi removida, por ser considerada irrelevante para o processo de classificação e não contribuir significativamente para o desempenho dos modelos.

Na sequência, foi realizada uma análise exploratória dos atributos categóricos presentes na base, especialmente nas colunas relacionadas ao protocolo de rede (*protocol_type*), serviço (*service*) e status da conexão (*flag*). Foram analisados os valores únicos de cada atributo, permitindo compreender melhor a distribuição das informações presentes na base. Como algoritmos de aprendizado de máquina supervisionado trabalham predominantemente com dados numéricos, foi necessário converter os atributos categóricos para representações numéricas. Para isso, foi utilizada a técnica de *Label Encoding*, disponibilizada pela biblioteca *Scikit-learn*, responsável por transformar cada categoria textual em um valor inteiro correspondente. Essa etapa foi fundamental para garantir a compatibilidade dos dados com os algoritmos utilizados posteriormente no treinamento dos modelos.

Além da transformação dos atributos categóricos, também foi realizado um processo de agrupamento das classes originais de ataques presentes na base *NSL – KDD*. Inicialmente, a base possui diversos tipos específicos de ataques, o que poderia aumentar a complexidade do problema de classificação. Dessa forma, os ataques foram reorganizados em categorias mais abrangentes, sendo elas [5] [13]:

- **Normal:** Tráfego legítimo e sem atividades maliciosas.
- **DoS (Denial of Service):** Ataques que visam sobrecarregar os recursos do sistema, tornando-o indisponível para usuários legítimos.
- **Probe:** Ataques de sondagem que buscam coletar informações sobre o sistema ou rede, identificando vulnerabilidades.
- **R2L (Remote to Local):** Ataques onde o invasor tenta obter acesso local a partir de uma máquina remota, explorando falhas de autenticação.
- **U2R (User to Root):** Ataques onde o invasor já possui acesso local e busca elevar seus privilégios para obter controle total do sistema.

Esse agrupamento teve como objetivo simplificar o processo de classificação, reduzir a complexidade do problema e melhorar a interpretação dos resultados obtidos pelos algoritmos de aprendizado de máquina.

²Disponível em: <https://www.kaggle.com/datasets/hassan06/nslddd>

³Arquivo disponível em: https://colab.research.google.com/drive/1ipTH1Lz002ih8L_m5bJHFtuD93UC1AQ?usp=sharing

C. Redução de Dimensionalidade

Após a etapa de pré-processamento dos dados, foi realizada uma análise exploratória com o objetivo de compreender a estrutura da base *NSL – KDD*, identificar padrões nos dados e verificar o comportamento das classes presentes no conjunto utilizado. Inicialmente, foram analisadas as dimensões da base de dados, os tipos de atributos existentes e a distribuição das classes de ataques. Essa etapa permitiu compreender a proporção entre registros classificados como tráfego normal e tráfego malicioso, além de identificar possíveis desbalanceamentos entre as categorias de ataques. Também foram geradas estatísticas descritivas dos atributos numéricos, permitindo observar medidas como média, desvio padrão, valores mínimos e máximos das variáveis presentes na base.

Além da análise estatística, foram construídas representações gráficas para auxiliar na interpretação dos dados. Foram utilizados gráficos de barras para visualização da distribuição das classes, possibilitando identificar quais categorias possuíam maior quantidade de registros. Posteriormente, foi aplicada uma análise de correlação entre os atributos numéricos da base utilizando o coeficiente de correlação de Pearson. Essa análise teve como principal objetivo identificar relações lineares entre as variáveis e detectar atributos altamente correlacionados, que poderiam representar informações redundantes no conjunto de dados. A matriz de correlação foi representada visualmente por meio de um heatmap, gerado com auxílio das bibliotecas Matplotlib e Seaborn, permitindo observar de forma mais intuitiva a intensidade das correlações entre os atributos.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Com o objetivo de reduzir a dimensionalidade da base e minimizar o impacto do elevado número de atributos no desempenho computacional dos modelos, foi aplicada a técnica de Análise de Componentes Principais (*PCA*). Antes da aplicação do *PCA*, os dados foram padronizados utilizando o método *StandardScaler*, responsável por transformar os atributos para média igual a zero e desvio padrão igual a um. Essa etapa foi necessária porque o *PCA* é sensível à escala dos dados, podendo atribuir maior relevância a atributos com amplitudes maiores. Após a padronização, o *PCA* foi aplicado para transformar os atributos originais em um novo conjunto de componentes principais, preservando a maior parte da variância existente nos dados.

$$X_{PCA} = W^T X$$

Durante essa etapa, foi realizada a análise da variância explicada acumulada dos componentes principais, permitindo identificar a quantidade ideal de

componentes necessários para representar adequadamente o conjunto de dados sem perda significativa de informação. Além disso, foram geradas visualizações bidimensionais utilizando os dois principais componentes obtidos pelo *PCA*, possibilitando observar o comportamento e a separação das classes presentes na base *NSL – KDD* em um espaço reduzido de dimensionalidade.

Por fim, também foi realizada uma etapa de seleção de atributos utilizando o algoritmo *DecisionTree*. O modelo foi empregado para calcular o grau de importância de cada variável durante o processo de classificação, permitindo identificar os atributos mais relevantes para o problema de detecção de intrusões. Com base nesses resultados, foi possível analisar quais características da base possuíam maior influência no desempenho dos modelos de aprendizado de máquina. Essa etapa contribuiu para redução de redundâncias, simplificação do conjunto de dados e melhoria do desempenho computacional dos algoritmos utilizados posteriormente no treinamento e avaliação dos modelos.

D. Treinamento dos Algoritmos

Nessa seção, foi realizada a etapa de treinamento dos algoritmos de aprendizado de máquina supervisionado. Essa fase teve como principal objetivo construir modelos capazes de identificar padrões presentes nos registros da base *NSL – KDD* e realizar a classificação do tráfego de rede entre conexões normais e diferentes categorias de ataques cibernéticos.

Inicialmente, os dados foram organizados em variáveis preditoras (features) e variável alvo (target). Os atributos da base foram utilizados como entrada dos algoritmos, enquanto as categorias correspondentes aos tipos de tráfego e ataques foram utilizadas como saída esperada dos modelos. Após essa definição, os dados passaram pela etapa de separação entre conjuntos de treinamento e teste utilizando a função `train_test_split` da biblioteca Scikit-learn, sendo o arquivo `KDDTrain+.txt` para treino e o `KDDTest+.txt` para teste, divididos na proporção clássica de 70% e 30%. Essa divisão permitiu utilizar parte dos registros para treinamento dos algoritmos e outra parte para validação do desempenho dos modelos em dados não utilizados durante o aprendizado. Essa abordagem foi fundamental para avaliar a capacidade de generalização dos algoritmos e reduzir problemas relacionados ao sobreajuste (*overfitting*).

Durante os experimentos, foram realizados treinamentos utilizando tanto o conjunto completo de atributos quanto versões reduzidas dos dados obtidas por meio da aplicação do *PCA* e da seleção de atributos baseada em importância das variáveis. Essa estratégia permitiu comparar o impacto da redução de dimensionalidade no desempenho dos modelos, bem como analisar possíveis melhorias relacionadas ao custo computacional e à eficiência das classificações.

Os algoritmos avaliados no treinamento foram:

Algoritmo	Tipo [6]	Definição [6]	Motivação
<i>DecisionTree</i>	Aprendizado Supervisionado (Classificação)	Cria regras lógicas (fluxograma) para dividir os dados e decidir.	Alta inter-pretabilidade, baixo custo computacional, bom para regras binárias.
<i>RandomForest</i>	Aprendizado Supervisionado (Ensemble/Bagging)	Combina múltiplas árvores de decisão para previsões mais precisas.	Robustez e alta acurácia, porém maior custo.
<i>KNN</i> (k=5)	Aprendizado Supervisionado (Proximidade)	Classifica baseando-se nos vizinhos mais próximos.	Simples, útil como baseline não paramétrico.
<i>LogisticRegression</i>	Aprendizado Supervisionado (Classificação Binária)	Calcula a probabilidade de eventos (0% a 100%).	Baseline clássico, rápido e interpretável.
<i>SVM</i> (kernel linear)	Aprendizado Supervisionado (Classificação)	Define o hiperplano que melhor separa grupos de dados.	Eficaz em alta dimensionalidade, porém mais lento.

Tabela I. Comparação de Algoritmos de Aprendizado de Máquina

Durante o treinamento, os algoritmos aprenderam padrões existentes nos registros da base *NSL – KDD*, relacionando características do tráfego de rede às categorias de ataques previamente definidas. Os modelos foram ajustados utilizando os dados previamente padronizados e transformados, garantindo maior estabilidade durante o processo de aprendizado. Além disso, foram realizados testes comparativos entre os modelos treinados com os atributos originais e os modelos treinados após aplicação das técnicas de redução e seleção de atributos, permitindo analisar os impactos dessas abordagens no desempenho final dos classificadores.

Para aumentar a confiabilidade dos resultados obtidos, também foi aplicada a técnica de validação cruzada (cross-validation) utilizando a função `cross_val_score` da biblioteca Scikit-learn. Essa técnica consiste em dividir os dados em múltiplas partições, permitindo que o algoritmo seja treinado e avaliado diversas vezes em subconjuntos diferentes da base de dados. Dessa forma, foi possível obter uma avaliação mais consistente e menos dependente de uma única divisão aleatória entre treino e teste.

Além das métricas de classificação, também foram analisados aspectos relacionados ao desempenho computacional dos modelos, como tempo de treinamento e eficiência do processamento após aplicação das técnicas de redução de dimensionalidade. Essa análise permitiu verificar não apenas

a capacidade dos algoritmos em detectar ataques, mas também sua viabilidade computacional para utilização em sistemas de detecção de intrusão, nos quais rapidez e eficiência são fatores relevantes para o monitoramento do tráfego de rede.

E. Avaliação de Desempenho

Após a etapa de treinamento dos algoritmos de aprendizado de máquina, foi realizada a avaliação dos modelos com o objetivo de medir a eficiência dos classificadores na detecção de intrusões presentes na base *NSL – KDD*. Durante os testes, foram comparados os resultados obtidos pelos modelos treinados com os atributos originais da base de dados, pelos modelos treinados após aplicação da técnica de redução de dimensionalidade *PCA* e pelos modelos treinados utilizando atributos selecionados com base na importância das variáveis calculadas pelo algoritmo *DecisionTree*. Essa comparação permitiu analisar o impacto das técnicas de redução e seleção de atributos tanto no desempenho dos classificadores quanto na eficiência computacional do processamento.

Para análise dos resultados, foram utilizadas métricas amplamente empregadas em problemas de classificação supervisionada e sistemas de detecção de intrusão, incluindo Accuracy, Precision, Recall e F1-Score. Essas métricas foram calculadas com auxílio das funções disponibilizadas pela biblioteca Scikit-learn, permitindo avaliar diferentes aspectos do desempenho dos algoritmos.

A métrica Accuracy foi utilizada para medir a proporção total de classificações corretas realizadas pelos modelos em relação ao total de registros analisados.

A métrica Precision foi empregada para avaliar a proporção de classificações positivas corretas em relação ao total de classificações positivas realizadas pelo modelo, sendo importante para análise da ocorrência de falsos positivos.

O Recall foi utilizado para medir a capacidade dos algoritmos em identificar corretamente os registros pertencentes às categorias de ataques, reduzindo a ocorrência de falsos negativos.

Já o F1-Score foi empregado para representar o equilíbrio entre precisão e recall, sendo especialmente relevante em problemas que apresentam desbalanceamento entre classes.

Durante a avaliação, também foram observados aspectos relacionados ao desempenho computacional dos modelos, incluindo tempo de treinamento, tempo de execução e impacto causado pela redução de dimensionalidade no processamento dos dados. Essa análise permitiu verificar se a aplicação das técnicas de *PCA* e seleção de atributos contribuiu para redução do custo computacional sem comprometer significativamente o desempenho dos classificadores.

Com base nos resultados obtidos, foi realizada uma comparação entre os algoritmos utilizados considerando métricas de desempenho, capacidade de generalização e eficiência computacional. O algoritmo *DecisionTree* apresentou boa interpretabilidade e capacidade de classificação, além de possibilitar a identificação dos atributos mais relevantes para o problema. Entretanto, o *RandomForest* demonstrou maior robustez e estabilidade nas previsões, devido à combinação de múltiplas árvores de decisão. A seleção do modelo final considerou o equilíbrio entre as métricas de Accuracy, Precision, Recall e F1-Score, além do impacto das técnicas de redução de dimensionalidade no desempenho e no custo computacional dos algoritmos, permitindo identificar a abordagem mais eficiente para classificação do tráfego de rede e detecção de intrusões utilizando a base *NSL - KDD*.

V. Resultados Preliminares

O algoritmo utilizado para as análises foi o *DecisionTree*, este conta com o menor tempo de teste e melhor acurácia, estes valores são a maior métrica utilizada pois o objetivo do sistema é ser executado em tempo real para garantir uma resposta rápida a incidentes.

Utilizando os testes do capítulo anterior, foi desenvolvida uma análise com os variados algoritmos mais utilizados (*RandomForest*, *DecisionTree*, *KNN*, *SVM*, *LogisticRegression*). Utilizando um random state de 42 para os algoritmos citados, obtiveram-se os seguintes valores:

Modelo	Acc ⁴	Prec ⁵	Rec ⁶	F1 ⁷	Tempo Treino(s)	Tempo T
RF	0.9987	0.9992	0.9979	0.9985	6.3216	0.120
DT	0.9977	0.9975	0.9975	0.9975	0.8541	0.000
KNN	0.9935	0.9921	0.9940	0.9931	0.0052	16.56
LR	0.9454	0.9504	0.9313	0.9408	3.0691	0.000
SVM	0.9544	0.9594	0.9418	0.9506	217.7609	9.050

Tabela II. Estatísticas de treino dos modelos

Dados os valores da tabela II o algoritmo selecionado foi o *DecisionTree*, dado seu baixo tempo de teste e alta acurácia.

Utilizando os valores anteriormente definidos na figura 6 para uma melhor visualização dos dados que realmente importam, o gráfico foi dividido entre ataques e tráfego normal:

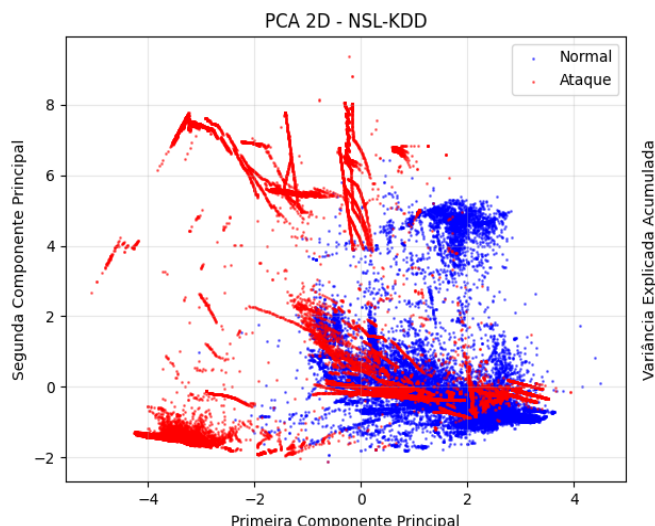


Figura 2. PCA classificando tráfego como ataque e normal

Estudando as features com o auxílio dos resultados do *PCA* chegou-se a Conclusão que com 7 features é possível alcançar uma acurácia com valores a cima de 95%:

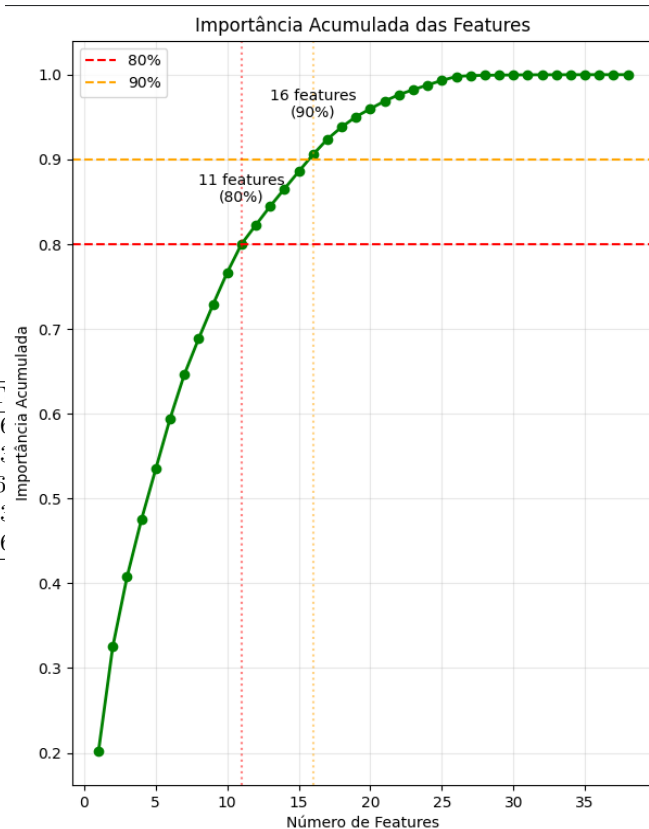


Figura 3. Representação da importância de cada feature para a acurácia

Afim de refinar os dados representado no *PCA* pela imagem 3 uma análise similar foi feita utilizando

o algoritmo *DecisionTree*:

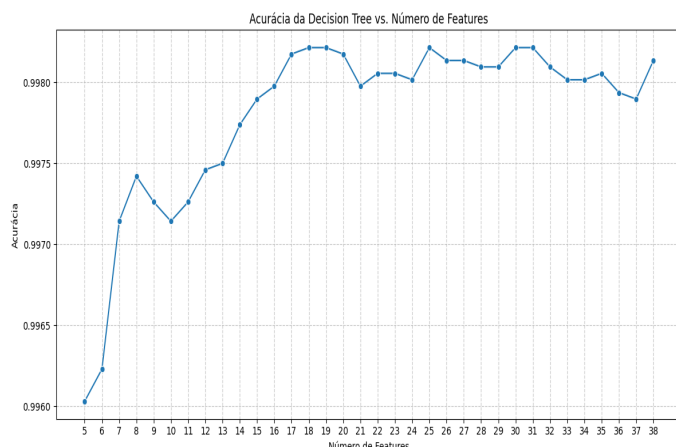


Figura 4. Features variadas para o decision tree

Indicando que o número ótimo de features é 18, onde a acurácia é maior. Sendo elas:

1. src.bytes
2. dst_host_srv_count
3. count
4. dst.bytes
5. dst_host_srv_error_rate
6. hot
7. dst_host_same_src_port_rate
8. logged_in
9. srv_count
10. dst_host_same_srv_rate
11. duration
12. wrong_fragment
13. dst_host_rerror_rate
14. dst_host_diff_srv_rate
15. dst_host_count
16. dst_host_srv_diff_host_rate
17. dst_host_serror_rate
18. diff_srv_rate

1. **src.bytes**: Bytes enviados da origem para o destino.
2. **dst_host_srv_count**: Número de conexões destinadas a mesma porta no host de destino.
3. **count**: Número de conexões feitas ao mesmo host de destino.
4. **dst.bytes**: Bytes enviados do destino de volta ao host de origem.
5. **dst_host_srv_error_rate**: Porcentagem de pacotes recusados tentando acessar a mesma porta do destino.
6. **hot**: Número de operações suspeitas detectadas na conexão.

7. **dst_host_same_src_port_rate** : Porcentagem de conexões para o destino que utilizaram a mesma porta de origem.

8. **logged_in** : Flag booleana que informa se o usuário autenticou no serviço ou não.

9. **srv_count**: Número de conexões feitas para a mesma porta que a conexão atual.

10. **dst_host_same_srv_rate** : Porcentagem de conexões que foram para a mesma porta do destino.

11. **duration** : Tempo de duração da conexão em segundos.

12. **wrong_fragment** : Número de fragmentos corrompidos na conexão.

13. **dst_host_rerror_rate** : Porcentagem de conexões que foram recusadas.

14. **dst_host_diff_srv_rate** : Porcentagem de conexões que foram para portas diferentes do destino.

15. **dst_host_count** : Número de conexões direcionadas para o mesmo ip de destino.

16. **dst_host_srv_diff_host_rate** : Porcentagem de conexões para a mesma porta que vieram de várias origens.

17. **dst_host_serror_rate** : Porcentagem de conexões que receberam erro do tipo *SYN*.

18. **diff_srv_rate** : Porcentagem de conexões que foram para portas diferentes.

A. Análise de Dados e Pré-processamento

Para a análise de dados foi considerado a base de dados NSL-KDD com os seguintes ataques listados na tabela III. alguns dos valores citados não são ataques reais, porém no contexto desta análise componentes como: *loadmodule*, *perl* e *nmap* foram categorizados como ataques.

normal	67343
neptune	41214
satan	3633
ipsweep	3599
portsweep	2931
smurf	2646
nmap	1493
back	956
teardrop	892
warezclient	890
pod	201
guess_passwd	53
buffer_overflow	30
warezmaster	20
land	18
imap	11
rootkit	10
loadmodule	9
ftp_write	8
multihop	7
phf	4
perl	3
spy	2

Tabela III. Dados presentes na base *NSL – KDD*

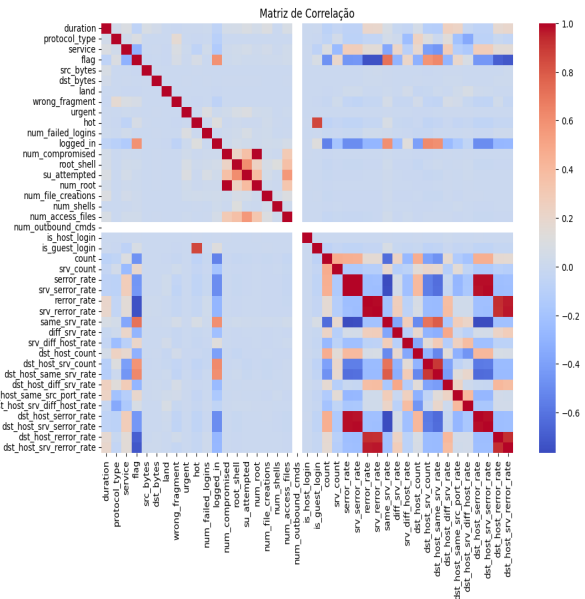
Para melhor visualização dos dados os ataques foram separados nas seguintes categorias: *normal*, *dos*, *probe*, *r2l*, *u2r*.

Categoria	Tipos de Ataque
normal	Normal
dos	Neptune smurf teardrop pod land
probe	nmap ipsweep satan
r2l	warezclient warezmaster guess_passwd imap ftp_write multihop phf spy
u2r	buffer_overflow rootkit loadmodule perl

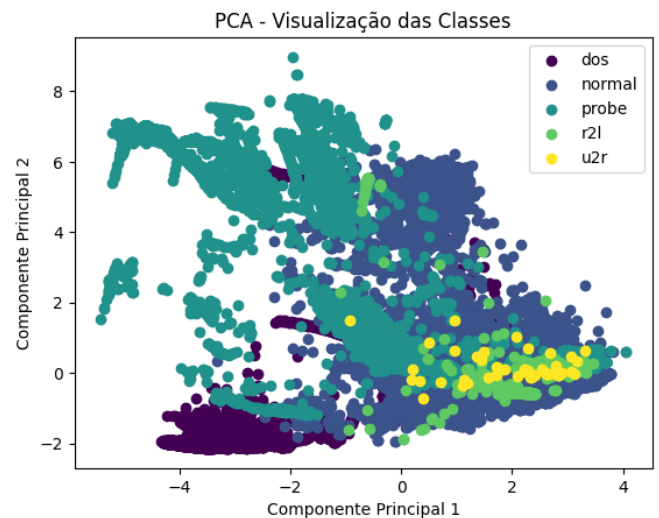
Tabela IV. Definição de tipos de ataque por grupos

Com o objetivo de analisar os dados e diminuir o espaço dimensional, foi utilizado a matriz de correlação 5 que representa que a contagem de pacotes

e ocorrência de erros são os componentes com maior correlação entre si.

Figura 5. Matriz de correlação da base *NSL_KDD*

Para validar as informações da matriz de correlação foi feita uma análise baseada em *PCA*, plotando um gráfico com os dois primeiros componentes e separando os valores pelos grupos anteriormente mencionados na tabela IV.

Figura 6. visualização de dois componentes do *PCA* separado por classes

B. Redução de Dimensionalidade e Seleção de Atributos

Buscando uma margem de variância a cima de 95% foi calculado o *PCA* para variados números de componentes, para descobrir a melhor variância, com o menor número de componentes. Definido 23 de todos os 43 componentes como a melhor média amostral

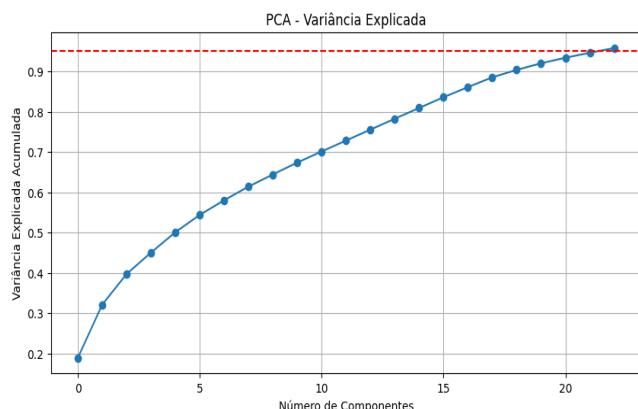


Figura 7. Variância explicada para PCA

VI. Considerações Finais e Perspectivas Futuras

Vai vir uma primeira parte aqui

A. Cronograma

- 1. Levantamento Inicial da Área de Pesquisa:** Realização de estudos introdutórios sobre a área de Segurança da Informação, com foco em Intrusion Detection Systems (IDS) e aplicação de técnicas de Machine Learning na detecção de ameaças cibernéticas.
- 2. Continuidade do Levantamento Teórico da Área:** Aprofundamento da pesquisa introdutória, visando consolidar o entendimento sobre redes de computadores, segurança cibernética e métodos de detecção de intrusões.
- 3. Definição do Tema de Pesquisa:** Delimitação do escopo do Trabalho de Conclusão de Curso, estabelecendo o foco na utilização de técnicas de aprendizado de máquina aplicadas à detecção de intrusões em redes computacionais.
- 4. Entrega do Checklist do TCC:** Organização e submissão da documentação inicial exigida para formalização e acompanhamento do desenvolvimento do Trabalho de Conclusão de Curso.
- 5. Pesquisa e Seleção de Referências Bibliográficas:** Levantamento de artigos científicos, trabalhos acadêmicos e materiais técnicos relacionados à detecção de intrusões, pré-processamento de dados e algoritmos de Machine Learning aplicados à cibersegurança.
- 6. Levantamento e Seleção das Bases de Dados:** Análise e escolha das bases de dados utilizadas no projeto, incluindo a NSL-KDD para o desenvolvimento inicial, além das bases UNSW-NB15 e CICIDS2017 previstas para utilização em etapas futuras do MVP.
- 7. Estudo de Técnicas de Pré-Processamento e Extração de Características:** Pesquisa sobre métodos de tratamento e redução de dimensionalidade dos dados, incluindo técnicas como PCA (Principal Component Analysis), matriz de correlação e extração de features relevantes para otimização dos modelos.
- 8. Pesquisa sobre Algoritmos de Machine Learning:** Estudo dos principais algoritmos supervisionados e não supervisionados aplicáveis à detecção de intrusões, analisando características, vantagens e limitações de cada abordagem.
- 9. Levantamento de Dados Analíticos para Fundamentação do Tema:** Coleta e análise de informações estatísticas e estudos recentes que evidenciam o crescimento das ameaças cibernéticas e a relevância da utilização de sistemas inteligentes de detecção de intrusão.
- 10. Elaboração das Referências Bibliográficas:** Estruturação e organização das referências utilizadas no trabalho, seguindo as normas acadêmicas exigidas.
- 11. Desenvolvimento Inicial Utilizando a Base NSL-KDD:** Realização de estudos práticos envolvendo pré-processamento de dados, aplicação de algoritmos supervisionados e análise de métricas de desempenho, como acurácia, precisão, recall e taxa de detecção.
- 12. Definição do Algoritmo Supervisionado Principal:** Escolha do algoritmo supervisionado mais adequado para o desenvolvimento do MVP1, considerando desempenho, capacidade de generalização e resultados obtidos nos testes preliminares.
- 13. Continuidade do Desenvolvimento com a Base NSL-KDD:** Aprimoramento das etapas de pré-processamento, treinamento e avaliação dos modelos supervisionados aplicados à base de dados NSL-KDD.
- 14. Sistematização da Revisão Bibliográfica:** Organização e consolidação dos conteúdos teóricos levantados ao longo da pesquisa, estruturando a fundamentação teórica do trabalho.
- 15. Sistematização dos Resultados do MVP1:** Documentação e análise dos resultados obtidos durante o desenvolvimento do primeiro MVP, destacando métricas de desempenho, limitações e possibilidades de melhoria.
- 16. Elaboração da Apresentação do Trabalho de Conclusão de Curso:** Desenvolvimento do material de apresentação contendo os principais objetivos, metodologia, resultados parciais e contribuições do projeto.

17. **Apresentação do Trabalho de Conclusão de Curso:** Apresentação formal do projeto desenvolvido, abordando fundamentação teórica, metodologia aplicada, desenvolvimento do MVP, resultados alcançados e considerações finais do trabalho.

Item	2026											
	Jan	Fev	Mar	Abr	Mai	Jun	Jul	Ago	Set	Out	Nov	Dez
1	X											
2		X										
3		X										
4		X										
5			X									
6			X									
7			X									
8			X									
9				X								
10				X								
11				X								
12					X							
13					X							
14					X							
15					X							
16					X							
17						X						

Tabela V. Cronograma do projeto

VII. Referências

- [1] E. E. Abdallah, W. Eleisah e A. F. Otoom, "Intrusion Detection Systems using Supervised Machine Learning Techniques: A survey," 2022. DOI: 10.1016/2022.03.029
- [2] O. H. Abdulganiyu, T. A. Tchakoucht e Y. K. Saheed, "A systematic literature review for network intrusion detection system (IDS)," *International Journal of Information Security*, vol. 22, pp. 1233–1262, 2023. DOI: 10.1007/s10207-023-00682-2
- [3] W. S. Admass, Y. Y. Munaye e A. A. Diro, "Cyber security: State of the art, challenges and future directions," 2023. DOI: 10.1016/2023.10031
- [4] Z. Azam, M. M. Islam e M. N. Huda, "Comparative Analysis of Intrusion Detection Systems and Machine Learning-Based Model Analysis Through Decision Tree," 2023. DOI: 10.1109/2023.3296444
- [5] M. C. Belavagi e B. Muniyal, "Performance Evaluation of Supervised Machine Learning Algorithms for Intrusion Detection," *Procedia Computer Science*, vol. 89, pp. 117–123, 2016. DOI: 10.1016/j.procs.2016.06.016
- [6] I. Belcic e C. Stryker, *O que é aprendizado supervisionado?* 2026. URL: <https://www.ibm.com/br-pt/think/topics/supervised-learning>
- [7] O. Depren, M. Topallar, E. Anarim e M. K. Ciliz, "An intelligent intrusion detection system (IDS) for anomaly and misuse detection in computer networks," *Expert Systems with Applications*, vol. 29, n.º 4, pp. 713–722, 2005. DOI: 10.1016/j.eswa.2005.05.002
- [8] S. Duquea e D. N. bin Omar, "Using Data Mining Algorithms for Developing a Model for Intrusion Detection System (IDS)," 2015. DOI: 10.1016/j.procs.2015.09.145
- [9] J. G., W. D., H. T., T. R. e T. J., *An Introduction to Statistical Learning with Applications in Python*. 2023. DOI: 10.1007/978-3-031-38747-0_2
- [10] X. Gao, C. Shan, C. Hu, Z. Niu e Z. Liu, "An Adaptive Ensemble Machine Learning Model for Intrusion Detection," 2019. DOI: 10.1109/ACCESS.2019.2923640
- [11] D. Ghelani, "Cyber Security, Cyber Threats, Implications and Future Perspectives: A Review," 2022. DOI: 10.11648/j.XXXX.2022XXXX.XX
- [12] F. Iglesias e T. Zseby, "Analysis of network traffic features for anomaly detection," 2014. DOI: 10.1007/10994-014-5473-9
- [13] J. Jabez e B. Muthukumar, "Intrusion Detection System (IDS): Anomaly Detection Using Outlier Detection Approach," *Procedia Computer Science*, vol. 48, pp. 338–343, 2015. DOI: 10.1016/j.procs.2015.04.191
- [14] A. Khraisat, I. Gondal, P. Vamplew e J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," *Cybersecurity*, vol. 2, n.º 1, pp. 1–22, 2019. DOI: 10.1186/s42400-019-0038-7
- [15] A. Khraisat, I. Gondal, P. Vamplew, J. Kamruzzaman e A. Alazab, "Hybrid Intrusion Detection System Based on the Stacking Ensemble of C5 Decision Tree Classifier and One Class Support Vector Machine," *Electronics*, vol. 8, n.º 9, p. 1011, 2019. DOI: 10.3390/electronics8091011
- [16] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin e K.-Y. Tung, "Intrusion detection system: A comprehensive review," *Journal of Network and Computer Applications*, vol. 36, n.º 1, pp. 16–24, 2013. DOI: 10.1016/j.jnca.2012.09.004
- [17] S. M. Othman, F. M. Ba-Alwi, T. Nabeel e A. Y. Al-Hashida, "Intrusion detection model using machine learning algorithm on Big Data environment," *Journal of Big Data*, vol. 5, n.º 1, p. 34, 2018. DOI: 10.1186/s40537-018-0145-4

- [18] M. M. Rathore, A. Ahmad e A. Paul, “Real time intrusion detection system for ultra-high-speed big data environments,” *The Journal of Supercomputing*, vol. 72, n.º 9, pp. 3489–3510, 2016. DOI: 10.1007/s11227-015-1615-5
- [19] T. Saranya, S. Sridevi, C. Deisy, T. D. Chung e M. K. A. A. Khan, “Performance Analysis of Machine Learning Algorithms in Intrusion Detection System: A Review,” *Procedia Computer Science*, vol. 171, pp. 1251–1260, 2020. DOI: 10.1016/j.procs.2020.04.133
- [20] S. A. R. Shah e B. Issac, “Performance comparison of intrusion detection systems and application of machine learning to Snort system,” *Future Generation Computer Systems*, vol. 80, pp. 157–170, 2018. DOI: 10.1016/j.future.2017.10.016
- [21] K. A. Taher, B. M. Y. Jisan, Md. e M. Rahman, “Network Intrusion Detection using Supervised Machine Learning Technique with Feature Selection,” 2019. DOI: 10.1109/2019.0644161
- [22] A. Thakkar e R. Lohiya, “A survey on intrusion detection system: feature selection, model, performance measures, application perspective, challenges, and future research directions,” *Artificial Intelligence Review*, vol. 54, pp. 4529–4593, 2021. DOI: 10.1007/s10462-021-10037-9
- [23] P. Vanin et al., “A Study of Network Intrusion Detection Systems Using Artificial Intelligence/Machine Learning,” 2022. DOI: 10.3390/app122211752

VIII. Anexos

IX. Agradecimentos

Agradecemos, primeiramente, às nossas famílias, pelo apoio incondicional, incentivo constante e compreensão ao longo de toda a nossa trajetória acadêmica. Sem esse suporte, a realização deste trabalho não seria possível.

Como inspiração para seguir em frente diante dos desafios, destacamos a frase de Walt Disney: “Todos os seus sonhos podem se tornar realidade se você tiver a coragem para persegui-los”.

Título	Citações	Autor	Ano	Tipo	HIDS/NIDS	Sig.	Técnica	Técnica Utilizada	Desempenho	Base
[23]	172	Vanin et al.	2022	Revisão	Ambos	Ambos	Ambos	CNN, DNN, SVM, Ensembles	99,24% (Acurácia)	KDD99, NSL-KDD, UNSW-NB15, CI-CIDS2017
[22]	459	Thakkar & Lohiya	2022	Pesquisa/Revisão	NIDS	Ambos	Ambos	ML, DL e SWEVO	95,5% (Acurácia)	KDD99, NSL-KDD, CI-CIDS2017
[2]	330	Abdulganiyu et al.	2022	SLR	NIDS	Ambos + Híbrido	Sup.	CNN, ANN, RF, SVM, DT	CNN+AE (100%)	KDD99, NSL-KDD, UNSW-NB15
[7]	803	Depren et al.	2005	Proposta	NIDS	Híbrido	Ambos	SOM (N. Sup.) e J.48 (Sup.)	99,84% (Uso Indevido)	KDD Cup 99
[10]	696	Gao et al.	2019	Proposta	NIDS	Anomalia	Supervisionada	Ensemble (DT, RF, KNN, DNN)	85,2% (Acurácia)	NSL-KDD
[12]	293	Iglesias & Zaeby	2015	Análise	NIDS	Anomalia	Supervisionada	DTC, kNN, NB, ANN, SVM	78,68% (Acurácia)	NSL-KDD
[4]	319	Azam et al.	2023	Revisão	NIDS	Híbrido	Supervisionada	DT (C4.5, ID3, CART)	97,36% (Acurácia)	NSL-KDD, ADEFA, KDD99
[11]	253	Ghelani et al.	2022	Visão Geral	NIDS/Ciberseg.	Geral	Plataforma IA	Geral ML/DL	Qualitativo	Bibliográfica
[3]	466	Admass et al.	2024	Revisão	Geral	Geral	Ambos	ML/DL	Síntese de Estado da Arte	EHR (Tempo Real)
[15]	256	Khraisat et al.	2020	Proposta	Ambos	Híbrido	Supervisionada	Stacking (C5.0 + One-Class SVM)	97,04% (ADFA)	NSL-KDD, ADFA
[13]	365	Jabez & Muthukumar	2015	Proposta	NIDS	Anomalia	Não Sup.	NOF (Outlier Detection)	Tempo de CPU superior	KDD Cup 99
[17]	305	Othman et al.	2018	Proposta	NIDS	Anomalia	Supervisionada	SVMWithSGD (Spark)	99,55% (AUROC)	KDD Cup 99
[16]	2463	Liao et al.	2013	Revisão	Vários	AD, SD, SPA	Vários	Bayes, Game Theory, Data Mining	Alta Acurácia (em SVM/DM)	KDD Cup 99
[21]	327	Taher et al.	2019	Estudo de Caso	NIDS	Anomalia	Supervisionada	SVM e ANN	94,02% (Acurácia)	NSL-KDD
[19]	555	Saranya et al.	2020	Revisão/Prop.	Ambos	Ambos + Hib.	Supervisionada	LDA, CART, RF	99,81% (Acurácia)	KDD-CUP
[20]	332	Shah & Issac	2018	Estudo Comp.	NIDS (Snort)	Híbrido	Supervisionada	SVM, DT, Fuzzy, Naive Bayes	SVM superior	Snort logs, DARPA, NSL-KDD
[5]	482	Belavagi & Muniyal	2016	Estudo de Caso	NIDS	Anomalia/Usu Ind.	Supervisionada	LR, SVM, GNB, RF	99% (Acurácia)	NSL-KDD
[18]	147	Rathore et al.	2017	Proposta	NIDS	Anomalia	Supervisionada	REPtree, J48, SVM, NB	99% (Taxa Ver. Pos.)	DARPA, KDD99, NSL-KDD
[14]	256	Khraisat et al.	2020	Proposta	Ambos	Híbrido	Supervisionada	Stacking (C5.0 + One-Class SVM)	97,04% (ADFA)	NSL-KDD, ADFA
[8]	175	Duque & Omar	2015	Proposta	NIDS context	Anomalia	Não Sup.	K-means (Clustering)	81,61% (Eficiência)	NSL-KDD
[1]	175	Abdallah et al.	2022	Pesquisa	NIDS focus	Anomalia/Usu Ind.	Supervisionada	RF, SVM, LR, GNB, J48, DNN	99,9% (Acurácia em RF)	KDD99, NSL-KDD, CI-CIDS2017
[9]	1879	James et al.	2023	Estudo Didático	Não se aplica	Não se aplica	Ambos	LR, DT, SVM, RF, KNN, PCA, K-means e outros	Não se aplica	Bases didáticas

Tabela VI. Revisão dos trabalhos sobre IDS